

SkillGap AI — Complete Project Documentation

Har ek Module, Algorithm, Preprocessing, aur Frontend-Backend Implementation ka Detailed Hinglish Breakdown

Project Name: SkillGap AI	Tech Stack: React + Vite, FastAPI, Scikit-Learn, SQLite
Author / Team: IDEA Lab	Coverage: 16 Tech Roles, 128 Skills, 36 Projects
Dataset: 8,000 Synthetic Samples	Production Model: Random Forest Classifier (88.4% Acc)
Date: March 2026	Language: Hinglish (Practical Explanations)

1. Project ka Aim aur Core Problem Kya Hai?

Asli Problem: Aaj kal students aur freshers ko pata nahi hota ki kisi specific job role (jaise AI Engineer, DevOps Engineer, Full Stack Developer) ke liye unke paas kaun si skills missing hain. Wo random courses karte rehte hain jisse time waste hota hai.

SkillGap AI ka Solution: Ye system student ka current skill set, unke kiye hue projects, target role aur experience level input leta hai. Fir ek **Trained Machine Learning Model (Random Forest)** ke zariye ye pata lagata hai ki candidate us role ke liye kitne percent ready hai (Match Percentage). Saath hi ek **Role-Skill Taxonomy** se missing skills find karta hai aur 3 stages (Foundational, Intermediate, Applied) mein **Visual Learning Roadmap** deta hai jise student track bhi kar sakta hai.

2. Project Folder Structure & File Responsibilities

Poora project 3 main layers mein divided hai: **Machine Learning (ml/)**, **Backend API (backend/)**, aur **Frontend (frontend/)**.

File / Folder Path	Ye File Kya Kaam Karti Hai? (Role in Project)
data/processed/role_skill_taxonomy.json	Taxonomy file: Isme 16 roles, 128 skills, har skill ka stage aur importance weight define hai.

data/processed/training_data.csv	8,000 synthetic student profiles ka dataset (current_skills, target_role, projects, label_fit).
ml/preprocessing.py	Synthetic data generator + Project-to-skill mapper + MultiLabelBinarizer aur LabelEncoders.
ml/train_model.py	Random Forest, SVM, Naive Bayes ko train karke benchmark karta hai aur .pkl artifacts save karta hai.
ml/model_artifacts/	Saved models: random_forest_model.pkl, skill_encoder.pkl, role_encoder.pkl, exp_encoder.pkl.
backend/main.py	FastAPI app entry point: CORS setup, SQLite database initialization, aur API routes registration.
backend/routes/	API endpoints: /roles (catalogue), /skillgap (ML prediction), /roadmap, /progress (tracking).
backend/services/prediction_service.py	Real-time ML Inference: Encoders load karta hai aur Random Forest se match % calculate karta hai.
backend/services/roadmap_service.py	Roadmap generator: Missing skills ko Foundational -> Intermediate -> Applied mein group karta hai.
frontend/src/pages/	React UI pages: Home (landing & stats), Dashboard (gap analysis), Explore (roles catalogue).
frontend/src/components/	Reusable UI: ProfileForm, MatchGauge (animated radial gauge), VisualRoadmap, ProgressTracker.

3. Role-Skill Taxonomy & Data Foundation

Taxonomy is project ka brain hai. Ye decide karti hai ki kaun si role ke liye kya-kya zaroori hai. Ye data `role_skill_taxonomy.json` mein store hota hai.

Har Role ke andar kya hota hai:

- **Skills List:** Har skill ka naam (e.g. 'Docker', 'PyTorch', 'SQL').
- **Stage:** Teen stages hoti hain — *Foundational* (Must-have basic), *Intermediate* (Core engineering), *Applied* (Production/Advanced tool).
- **Weight:** 0.3 se lekar 1.0 tak importance score. Core skill ka weight 0.8-1.0 hota hai, jabki optional ka 0.3-0.5.

Category	Roles Covered (Total 16 Roles)	Key Benchmark Skills
AI & Data	Data Scientist, ML Engineer, AI Engineer, Data Engineer	Python, PyTorch, RAG, LLM Fine-Tuning, Spark, Airflow, Scikit-learn
Software & Web	Backend Developer, Frontend Developer, Full Stack, Java Developer	React, Node.js, Spring Boot, REST APIs, Microservices, TypeScript
Cloud & DevOps	DevOps Engineer, Cloud Engineer, Cybersecurity Analyst	Kubernetes, Docker, Terraform, AWS, SIEM, Wireshark, Linux/Bash
Specialized	Mobile App, QA Automation, Blockchain, Game, Embedded Systems	Flutter, Cypress, Solidity, Smart Contracts, Unity/C#, C/C++, FreeRTOS

Project-to-Skill Implicit Deduction (Smart Inference):

Real life mein jab student bolta hai ki usne '*Kubernetes CI/CD Pipeline*' banaya hai, toh iska matlab use Docker, CI/CD, Linux/Bash, aur Kubernetes pehle se aate hain. Humara `PROJECT_TO_SKILLS` dictionary 36 projects ko map karke unke underlying skills automatically student ke profile mein include kar leta hai.

4. Dataset Kaise Bana? (Synthetic Generation & Ground Truth)

Real student data ka lack of label issue solve karne ke liye `generate_synthetic_dataset(n_samples=8000)` banaya gaya.

Step-by-step Generation Logic:

1. **16 Roles x 500 Samples = 8,000 Rows:** Har role ke barabar samples generate hote hain taaki class imbalance na ho.
2. **Experience Distribution:** 35% Beginner, 40% Intermediate, 25% Advanced.
3. **Weighted Skill Sampling:** High-weight skills ke choose hone ki probability zyada hoti hai.
4. **Realistic Noise:** Har profile mein 0 se 3 random non-role skills bhi add hoti hain (real students ki tarah).
5. **Projects Assignment:** Har student ko 0 se 3 random projects assign hote hain.

Ground Truth Label Formula (Kaise decide hua ki student FIT hai ya nahi):

compute_fit_score() function niche diye formula se label 1 ya 0 assign karta hai:

Matched_Weight = Student ke paas jo role skills hain unke weights ka sum

Total_Weight = Us role ke sabhi required skills ke weights ka sum

Base_Score = Matched_Weight / Total_Weight

Exp_Bonus = Beginner (+0.00), Intermediate (+0.05), Advanced (+0.10)

Noise = Gaussian random noise $\sim N(0, 0.05)$ [Interviews ki uncertainty simulate karne ke liye]

Final Score = Base_Score + Exp_Bonus + Noise

Agar Final Score ≥ 0.55 -> label_fit = 1 (Ready), warna 0 (Gap Exists).

5. Feature Engineering: Machine Learning Input Matrix

Machine learning model raw strings (jaise 'Python|Docker') direct nahi padh sakta. Isliye preprocess_dataset() data ko mathematical form mein transform karta hai:

Step	Technique	Dimensions	Explanation (Hinglish)
1. Skills Vectorization	MultiLabelBinarizer	8,000 x 128	Sabhi 128 unique skills ka one-hot/multi-hot vector banta hai. Agar student ke paas skill hai toh column mein 1, nahi toh 0.
2. Target Role Encoding	LabelEncoder	8,000 x 1	16 roles ko 0 se 15 ke integer codes mein convert karta hai (e.g. AI Engineer=0, Backend=1, etc.).
3. Experience Level Encoding	LabelEncoder	8,000 x 1	Experience levels ko 0 (Beginner), 1 (Intermediate), 2 (Advanced) code deta hai.
4. Feature Stacking	np.hstack([X_skills, X_role, X_exp])	8,000 x 130	Final Feature Matrix X banti hai jisme 130 columns hote hain (128 skills + 1 role + 1 experience).

6. Model Training, Comparison aur Final Selection

8,000 samples ko **80% Train Set (6,400 rows)** aur **20% Test Set (1,600 rows)** mein split kiya gaya (stratify=y ke saath taaki class ratio maintain rahe). Humne 3 models ko benchmark kiya:

Model Name	Accuracy	Precision	Recall	F1-Score	Status & Verdict
Random Forest	88.44%	92.61%	86.62%	89.52%	DEPLOYED (Winner)
SVM (RBF Kernel)	83.56%	83.98%	87.94%	85.91%	Good, but lacks calibrated proba
Naive Bayes	68.87%	74.24%	69.52%	71.80%	Baseline (Assumes feature independence)

Kyun Random Forest ko Production Model chuna gaya?

- Non-linear Interactions:** Real life mein skills akele kaam nahi karti — jaise *React + TypeScript* ya *Docker + Kubernetes* ka combination bohot powerful hota hai. Decision trees aisi combinations ko natural split se sikh lete hain.
- predict_proba() Support:** Random Forest 200 decision trees ke votes ko average karke exact probability deta hai (e.g. 0.742). Yahi calibrated probability hamare frontend ka **74.2% Match Score Gauge** banati hai.
- High Precision (92.6%):** Jab model kisi ko 'Fit' bolta hai, toh 92.6% chances hain ki candidate sach me fit hai (false positive risk minimal hai).

7. Backend API Architecture (FastAPI + Services)

Backend **FastAPI** framework par bana hai jo bohot fast aur asynchronous execution provide karta hai. Isme 5 primary API endpoints hain:

Endpoint & Method	Input Request	Internal Service Logic	Output Response
GET /api/roles	None	_load_taxonomy() cached JSON read karta hai.	16 roles ki full list with skills & stages.
POST /api/skillgap	student_id, current_skills, target_role, projects, experience_level	prediction_service.predict(): 130-dim vector banata hai, predict_proba se match % nikalta hai, missing skills filter karta hai.	match_percentage (0-100%), missing_skills (with weights & stages).
POST /api/roadmap	Same profile payload	roadmap_service.generate_roadmap(): Missing skills ko Foundational, Intermediate, Applied mein group karta hai.	Stage-wise learning sequence with estimated effort.
POST /api/progress	student_id, role, skill_name, completed	SQLite database (progress.db) mein row insert ya update karta hai.	success: true, updated_score
GET /api/progress/{id}/{role}	Path parameters	Database se student ke completed skills fetch karta hai.	List of completed skills for that role.

8. Frontend React Components & Dynamic UI

Frontend React 18 + Vite ke saath banaya gaya hai. Koi heavy CSS library nahi balki highly optimized, modern dark-mode vanilla CSS use ki gayi hai:

- **ProfileForm.jsx**: Target Role dropdown (sab 16 roles dynamic load karta hai). Skills aur Projects ke liye TagInput with live auto-suggestions dropdown.
- **MatchGauge.jsx**: Pure SVG-based radial circular progress meter jo 0% se target percentage tak smooth easeOutCubic transition animate karta hai.
- **SkillGapReport.jsx**: Present skills ko green badges aur missing skills ko weight indicators ke saath display karta hai.
- **VisualRoadmap.jsx**: 3-column stage layout (Foundational -> Intermediate -> Applied) jisme step-by-step milestones dikhte hain.
- **ProgressTracker.jsx**: Har missing skill ke aage interactive checkbox. User tick karta hai toh real-time SQLite update hota hai aur readiness meter badhta hai.
- **RolesCatalogue.jsx**: Explore page jisme 16 roles ke custom cards, emojis, descriptions aur skill distributions dikhte hain.

9. End-to-End Execution: Click Se Output Tak Kya Hota Hai?

Jab koi user website par aata hai aur apna gap analyze karta hai, toh background mein exact kya steps execute hote hain:

- Step 1: User Profile Fill Karta Hai:** User Target Role (e.g. 'DevOps Engineer') select karta hai, experience 'Intermediate' chunta hai, aur skills ('Linux/Bash', 'Docker') add karta hai.
- Step 2: Frontend API Call:** ProfileForm JSON payload banakar Axios client ke through POST /api/skillgap aur POST /api/roadmap ko parallel call karta hai.
- Step 3: Project Deductions:** Backend service dekhti hai agar student ne koi project mention kiya hai, toh uske underlying skills ko extract karke unified_skills list mein merge kar deti hai.
- Step 4: Vectorization & Encoding:** skill_encoder.pkl 128-dim binary array banata hai. role_encoder.pkl role ko integer code deta hai. Dono stack hokar 130-dim vector banate hain.
- Step 5: Random Forest Predicts:** random_forest_model.predict_proba(X) probability nikalta hai. Class 1 ka probability float multiply by 100 banakar match_percentage ban jata hai (e.g. 68.5%).
- Step 6: Gap & Roadmap Computation:** Taxonomy se role ki required skills compare hoti hain. Jo skills student ke paas nahi hain unhe Foundational, Intermediate, Applied stages mein arrange kiya jata hai.
- Step 7: UI Rendering & Tracking:** Frontend par Match Gauge animate hota hai, Gap analysis open hoti hai, aur Progress tracker activate ho jata hai.

10. Project Run, Retrain aur Test Karne Ke Commands

Project ko run aur manage karne ke liye ye standard commands use kiye jate hain:

1. Backend Start Karna (FastAPI on Port 8000):

```
cd SkillGap-AI
uvicorn backend.main:app --reload --port 8000
```

2. Frontend Start Karna (Vite Dev Server on Port 5173):

```
cd SkillGap-AI/frontend
npm run dev
```

3. Machine Learning Model Re-train Karna:

```
cd SkillGap-AI
python ml/train_model.py
```

Summary: Ye project sirf ek simple CRUD application nahi hai, balki ek complete **ML-integrated Career Intelligence Platform** hai jo realistic data synthesis, multi-label binarization, ensemble classification, aur dynamic reactive frontend ko seamlessly combine karta hai.